

Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Eş zamanlılık

- Bir programda birden fazla kontrol akışının aynı zaman aralığında ilerleyebilmesidir.
- Fiziksel eşzamanlılık: farklı işlemcilerde gerçekten paralel yürütme.
- Mantıksal eşzamanlılık: tek işlemcide zaman paylaşımıyla birlikte yürüyormuş gibi görünme.
- Dil tasarımı açısından soru: görevler nasıl tanımlanır, başlatılır, durdurulur ve haberleşir?
- Amaç:
 - Çok çekirdekli donanımdan yararlanmak.
 - G/Ç beklemelemlerini hesaplama ile örtüştürmek.
 - Arayüz, sunucu ve gerçek zamanlı sistemlerde tepki süresini iyileştirmek.
 - Doğal olarak bağımsız iş parçalarını daha okunabilir şekilde modellemek.

Fylnn Sınıflandırma

- SISD: tek komut akışı, tek veri akışı; klasik ardışık yürütme.
- SIMD: tek komut akışı, çoklu veri akışı; vektör ve veri-paralel hesaplama.
- MIMD: çoklu komut ve çoklu veri akışı; genel amaçlı çok işlemcili sistemler.
- Eşzamanlı programlama genellikle MIMD ve çok çekirdekli işlemcilerle ilişkilidir.
- Mantıksal eşzamanlılıkta görevler dönüşümlü çalışır; aynı anda ilerliyormuş izlenimi verir.
- Fiziksel eşzamanlılıkta görevler farklı işlemcilerde gerçek anlamda aynı anda yürür.
- Programlama dili desteği iki durumda da benzer soyutlamaları sunabilir.
- Hata türleri benzerdir: yarış durumu, kilitlenme, görünürlük ve zamanlama bağımlılığı.

İş Parçacığı

- Kontrol iş parçacığı, programın yürütme sırasını izleyen akıştır.
- Tek iş parçacıklı programda bir kontrol yolu vardır.
- Çok iş parçacıklı programda aynı adres alanını paylaşan birden çok akış bulunur.
- Paylaşılan bellek kolay iletişim sağlar; fakat senkronizasyon zorunludur.
- Bir program birden fazla görev veya thread içerir.
- Her görev kendi yürütme durumuna, program sayacına ve çalışma bağlamına sahiptir.
- Görevler aynı veri yapısına erişebilir ya da mesajlarla haberleşebilir.
- Temel tasarım sorusu: bağımsızlık, paylaşım ve senkronizasyon nasıl ifade edilir?

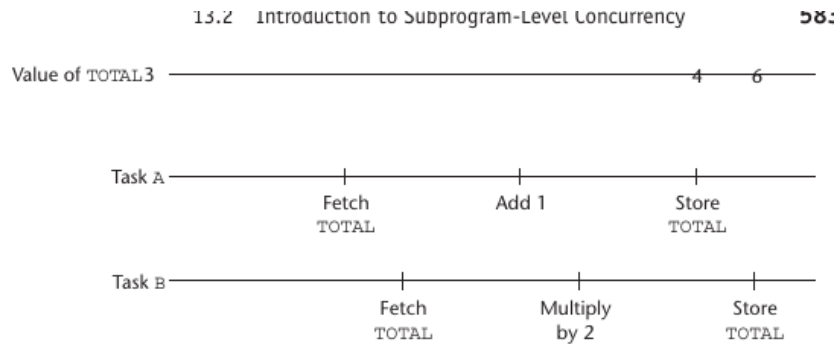
Yarışma Durumu

- İki görev aynı paylaşılan değişkene erişir.
- İşlemler atomik değilse son değer yürütme sırasına bağlı olur.
- Şekil TOTAL değişkeninde bu problemi gösterir.
- Çözüm: kritik bölgeye aynı anda yalnızca bir görev girmelidir.

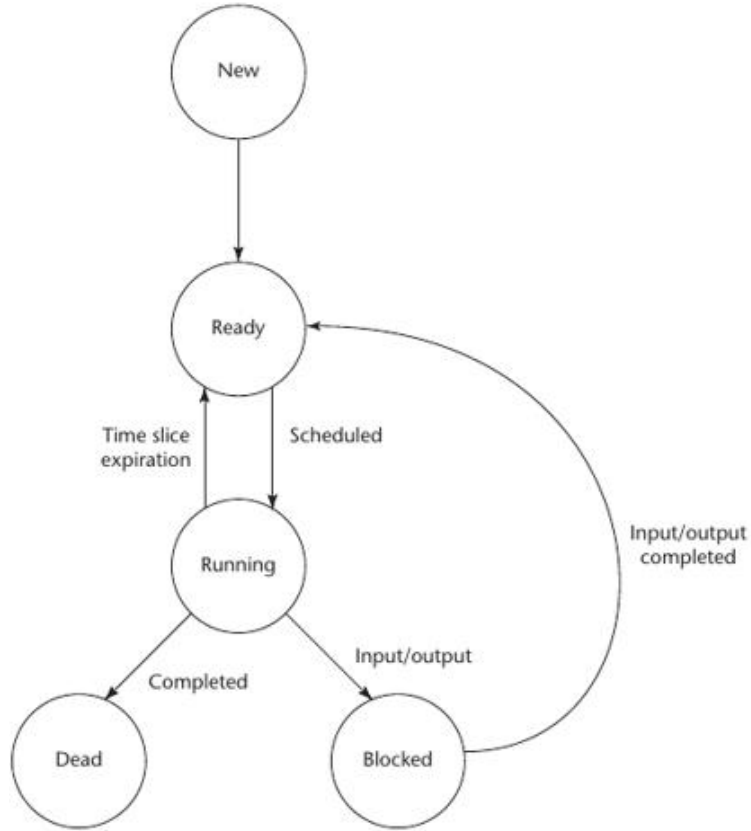
- Rekabet senkronizasyonu: ortak kaynağa erişimi karşılıklı dışlama ile düzenler.
- İşbirliği senkronizasyonu: bir görevin diğerinden veri/olay beklemesini sağlar.
- Üretici-tüketici problemi iki tür senkronizasyonu birlikte gösterir.
- Dil mekanizması hem güvenli erişim hem de bekleme/uyandırma sağlamalıdır.

Figure 13.1

The need for competition synchronization



Yaşam Döngüsü



- New: görev oluşturulmuştur fakat çalışmaya hazır değildir.
- Ready: çalışabilir durumdadır ve zamanlayıcıyı bekler.
- Running: işlemci üzerinde yürür.
- Blocked: G/Ç, mesaj veya senkronizasyon olayı bekler; Dead: yürütme tamamlanmıştır.
- Zamanlayıcı Ready durumundaki görevlerden birini Running durumuna geçirir.
- G/Ç isteği veya senkronizasyon bekleme Blocked durumuna geçiş yaratır.
- Zaman dilimi biterse görev tekrar Ready kuyruğuna döner.
- Tamamlanan görev Dead durumuna geçer.

Başlatma ve Semaphore

- Hangi hazır görev çalıştırılacak?
- Öncelik, zaman dilimi, adil sıra ve gerçek zamanlı kısıtlar kararı etkiler.
- Preemption varsa görev istenmeyen bir noktada kesilebilir.
- Bu nedenle paylaşılan veri işlemleri atomik kabul edilmemelidir.
- Semaphore, sayaç ve bekleyen görev kuyruğundan oluşan senkronizasyon yapısıdır.
- wait işlemi kaynak yoksa görevi bekletir.
- release işlemi kaynağı serbest bırakır ve bekleyen görevi uyandırabilir.
- İşlemler atomik olmalıdır; aksi halde semafor da yarış durumuna girer.

Semaphore

```
wait(aSemaphore)
if aSemaphore.counter > 0 then
    decrement aSemaphore.counter
else
    put the caller in aSemaphore.queue
    attempt to transfer control to a ready task

release(aSemaphore)
if aSemaphore.queue is empty then
    increment aSemaphore.counter
else
    put one waiting task into the ready queue
```

- Semaphore, sayaç ve bekleyen görev kuyruğundan oluşan senkronizasyon yapısıdır.
- wait işlemi kaynak yoksa görevi bekletir.
- release işlemi kaynağı serbest bırakır ve bekleyen görevi uyandırabilir.
- İşlemler atomik olmalıdır; aksi halde semafor da yarış durumuna girer.
- Düşük seviyeli ve güçlüdür; pek çok senkronizasyon örüntüsünü ifade eder.
- wait/release sırası programcıya bırakıldığı için hata olasılığı yüksektir.
- Yanlış sıra deadlock veya starvation doğurabilir.
- Büyük sistemlerde kodun okunabilirliği ve doğrulanabilirliği düşebilir.

Üretici-tüketici problemi

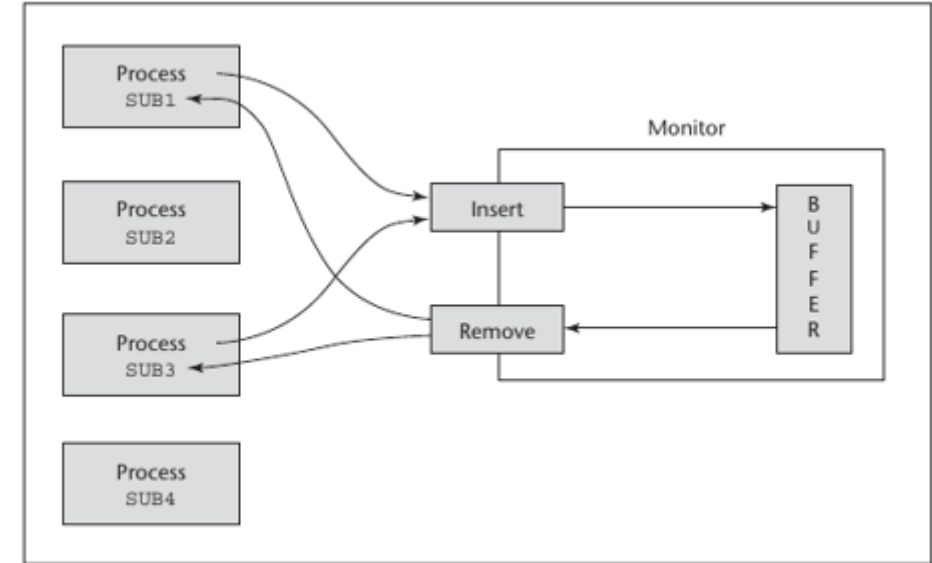
- Sınırlı tampon, üretici ve tüketici arasında paylaşılır.
- Üretici boş yer yoksa beklemelidir.
- Tüketici dolu yer yoksa beklemelidir.
- Tampona fiziksel erişim ayrıca karşılıklı dışlama gerektirir.

```
task producer;
loop
    -- yeni VALUE üret
    wait(emptyspots);    -- boş yer olduğundan emin ol
    wait(access);        -- tampon erişimi için kilit
al
    DEPOSIT(VALUE);
    release(access);     -- tamponu serbest bırak
    release(fullspots);  -- dolu yer sayısını artır
end loop;

task consumer;
loop
    wait(fullspots);     -- tampon boş değil mi?
    wait(access);        -- tampon erişimi için
kilit al
    FETCH(VALUE);
    release(access);     -- tamponu serbest bırak
    release(emptyspots); -- boş yer sayısını artır
    -- VALUE kullan
end loop;
```

Monitor

- Monitor, paylaşılan veri ve ona erişen işlemleri tek kapsül içinde toplar.
- Aynı anda monitor içinde yalnızca bir süreç/görev çalışabilir.
- Koşul değişkenleri işbirliği senkronizasyonu için bekleme/uyandırma sağlar.
- Soyut veri tipi gibi görünür; fakat otomatik karşılıklı dışlama içerir.

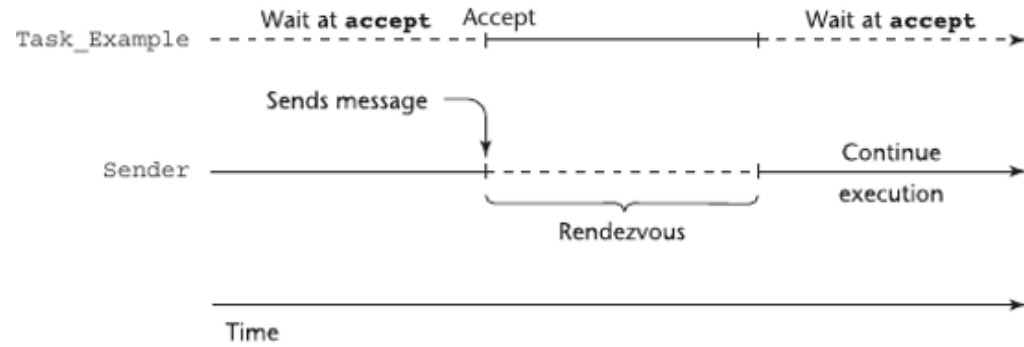


Mesajlaşma

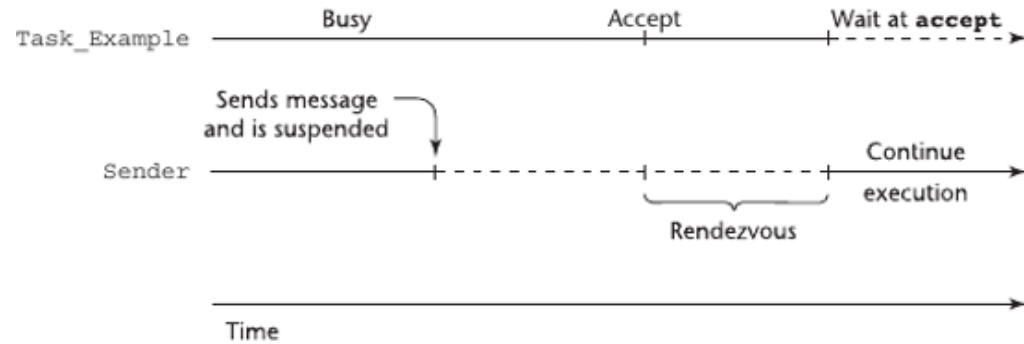
- Görevler paylaşılan bellek yerine mesajlarla iletişim kurar.
- send ve receive işlemleri veri aktarımı ve senkronizasyonu birlikte ifade edebilir.
- Dağıtık sistemlere ve paylaşımsız bellek mimarilerine daha uygundur.
- Soru: gönderme/alma senkron mu, asenkron mu?
- Senkron modelde gönderici ve alıcı aynı anda hazır olunca mesaj aktarılır.
- Bu buluşma anı Ada terminolojisinde rendezvous olarak adlandırılır.
- Asenkron modelde mesaj bir kuyrukta saklanabilir; gönderici beklemeyebilir.
- Senkron model akıl yürütmeyi kolaylaştırır; asenkron model gevşek bağılılık sağlar.

Ada Rendezvous

- Task A, Task B içindeki bir entry çağrısı yapar.
- Task B ilgili accept ifadesinde bekliyorsa çağrı gerçekleşir.
- Gönderici ve alıcı eşleştğinde rendezvous oluşur.

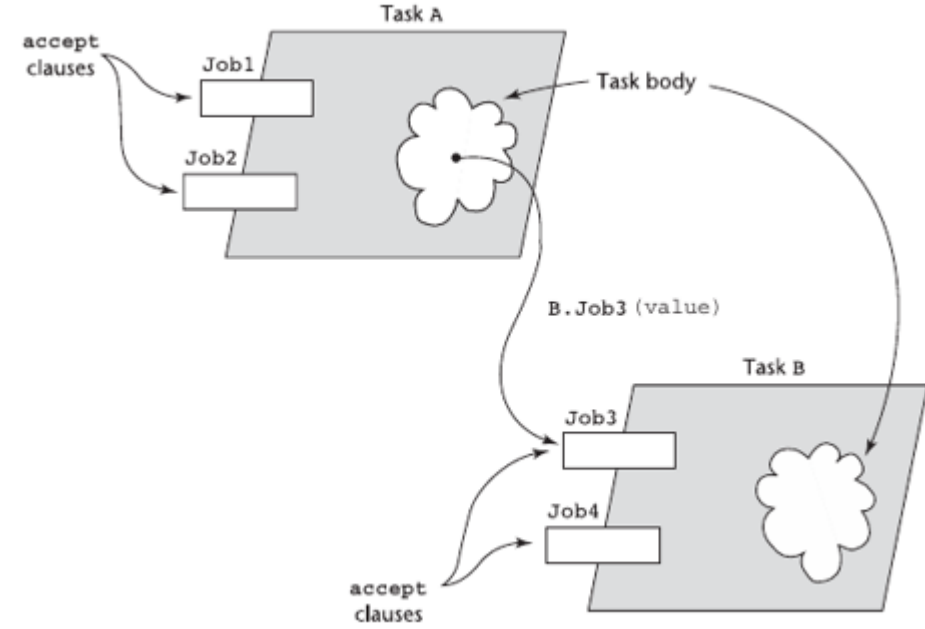


(a) Task_Example waits for Sender



Ada Rendezvous

- Task A, Task B içindeki Job3 entry noktasına mesaj gönderir.
- Task B kabul noktasına geldiğinde rendezvous tamamlanır.
- Accept clause yapısı iletişim arayüzünü belirginleştirir.



Java Threads

- Java threadleri aynı program adres alanında çalışır.
- Ada tasklerinden farklı olarak daha hafif birim olarak ele alınır.
- Thread oluşturma için Thread sınıfı genişletilebilir veya Runnable uygulanabilir.
- start(), run() metodunu yeni bir thread içinde yürütür.
- join(): çağıran thread, hedef thread bitene kadar bekleyebilir.
- sleep(): thread belirli süre askıya alınır.
- yield(): işlemciyi başka hazır threadlere bırakma önerisi verir.
- interrupt(): bekleyen/uyuyan threadin kesilmesini bildirmek için kullanılır.

```
• class MyThread extends Thread {  
    public void run() {  
        // thread tarafından  
        yürütülecek kod  
    }  
}  
  
MyThread myTh = new MyThread();  
myTh.start(); // run() ayrı kontrol  
akışında çalışır
```

Java Synchronized

- synchronized metod veya blok, nesne kilidi üzerinde karşılıklı dışlama sağlar.
- Bir synchronized metod çalışırken aynı nesnedeki diğer synchronized metodlar bekler.
- Kilidin kapsamı dikkatli seçilmelidir: fazla geniş kapsam paralelliği azaltır.
- wait(), notify(), notifyAll() işbirliği senkronizasyonu için kullanılır.

```
• class Queue {  
    private int[] que;  
    private int filled = 0;  
  
    public synchronized void deposit(int  
item) {  
        while (filled == que.length) wait();  
        // item kuyruk içine konur  
        filled++;  
        notifyAll();  
    }  
  
    public synchronized int fetch() {  
        while (filled == 0) wait();  
        // kuyruktan item alınır  
        filled--;  
        notifyAll();  
        return item;  
    }  
}
```

CSharp Threads

- C# içinde herhangi bir metod ayrı thread olarak yürütülebilir.
- Thread sınıfı System.Threading ad alanında bulunur.
- ThreadStart delegeesi çalışacak metodu işaret eder.
- Start(), Join(), Sleep(), Interrupt() gibi işlemler Java'ya benzer amaçlar taşır.

```
using System.Threading;

void DoWork() {
    // ayrı thread içinde çalışacak kod
}

Thread worker = new Thread(new
ThreadStart(DoWork));
worker.Start();
worker.Join();
```


CSharp Senkronizasyon

- lock deyimi kritik bölgeyi ifade etmek için kullanılır.
- Monitor sınıfı Enter, Exit, Wait, Pulse gibi işlemler sunar.
- Mutex ve Semaphore gibi daha düşük seviyeli yapılar da bulunur.
- .NET kitaplığı eşzamanlı koleksiyonlarla programlama yükünü azaltır.

Fonksiyonel Diller

- Fonksiyonel dillerde değişmez veri yapıları paylaşım hatalarını azaltabilir.
- Yan etkilerin sınırlı olması eşzamanlı akıl yürütmeyi kolaylaştırır.
- Bölüm MultiLisp, ML ve F# örnekleri üzerinden farklı yaklaşımları ele alır.
- Eşzamanlılık, saf fonksiyonel yapı ile çalışma zamanı desteğinin birleşimidir.
- MultiLisp, Scheme tabanlıdır ve eşzamanlı değerlendirme için future yapısını kullanır.
- future ifadesi bir hesaplamayı ayrı görev olarak başlatabilir.
- Değer gerektiğinde hesaplama bitmemişse bekleme gerçekleşir.
- Bu model paralelliği program metnine gömülü, nispeten yalın bir biçimde ifade eder.

Fonksiyonel Diller

- Fonksiyonel dillerde değişmez veri yapıları paylaşım hatalarını azaltabilir.
- Yan etkilerin sınırlı olması eşzamanlı akıl yürütmeyi kolaylaştırır.
- Bölüm MultiLisp, ML ve F# örnekleri üzerinden farklı yaklaşımları ele alır.
- Eşzamanlılık, saf fonksiyonel yapı ile çalışma zamanı desteğinin birleşimidir.
- F#, .NET thread sınıflarını kullanabilir.
- Ayrıca async workflow yapısıyla asenkron hesaplamayı yüksek seviyede ifade eder.
- MailboxProcessor/agent yaklaşımı mesaj temelli programlamaya uygundur.
- .NET concurrent collections F# programlarında da kullanılabilir.

Hatalar

- Paylaşılan değişkeni kilitlemeden okumak/yazmak.
- wait/release veya lock/unlock çiftlerini hatalı sırada kullanmak.
- Kilit kapsamını çok geniş tutarak paralelliği öldürmek.
- Deadlock olasılığını kilit edinme sırası üzerinden analiz etmemek.

Sorular

- Semaphore ile monitor arasındaki soyutlama farkını üretici-tüketici üzerinden açıklayın.
- Ada rendezvous modeli ile Java synchronized/wait-notify modelini karşılaştırın.
- Statement-level concurrency hangi durumlarda thread yazmaktan daha uygundur?